Mural de Publicações

Explicação didática do código, linha a linha

Node.js • Express • Handlebars • Sequelize • SQLite

VERSÃO AMPLIADA

Cada linha cumpre uma responsabilidade

Entrada
Execução
Resultado
Conexão

Como estudar com esta apresentação

- Leia primeiro a finalidade do arquivo.
- Observe somente a linha destacada.
- Diga com suas palavras o que ela realiza.
- Relacione a linha ao resultado visto no navegador.
- Retorne ao bloco completo ao final da funcionalidade.

O objetivo é compreender o percurso, e não decorar comandos.

O resultado que queremos explicar

- A página inicial confirma que o servidor funciona.
- A listagem recupera seis publicações do SQLite.
- O link abre os detalhes pelo identificador.
- O endpoint /saude responde em JSON.
- Um endereço inexistente produz uma página 404.

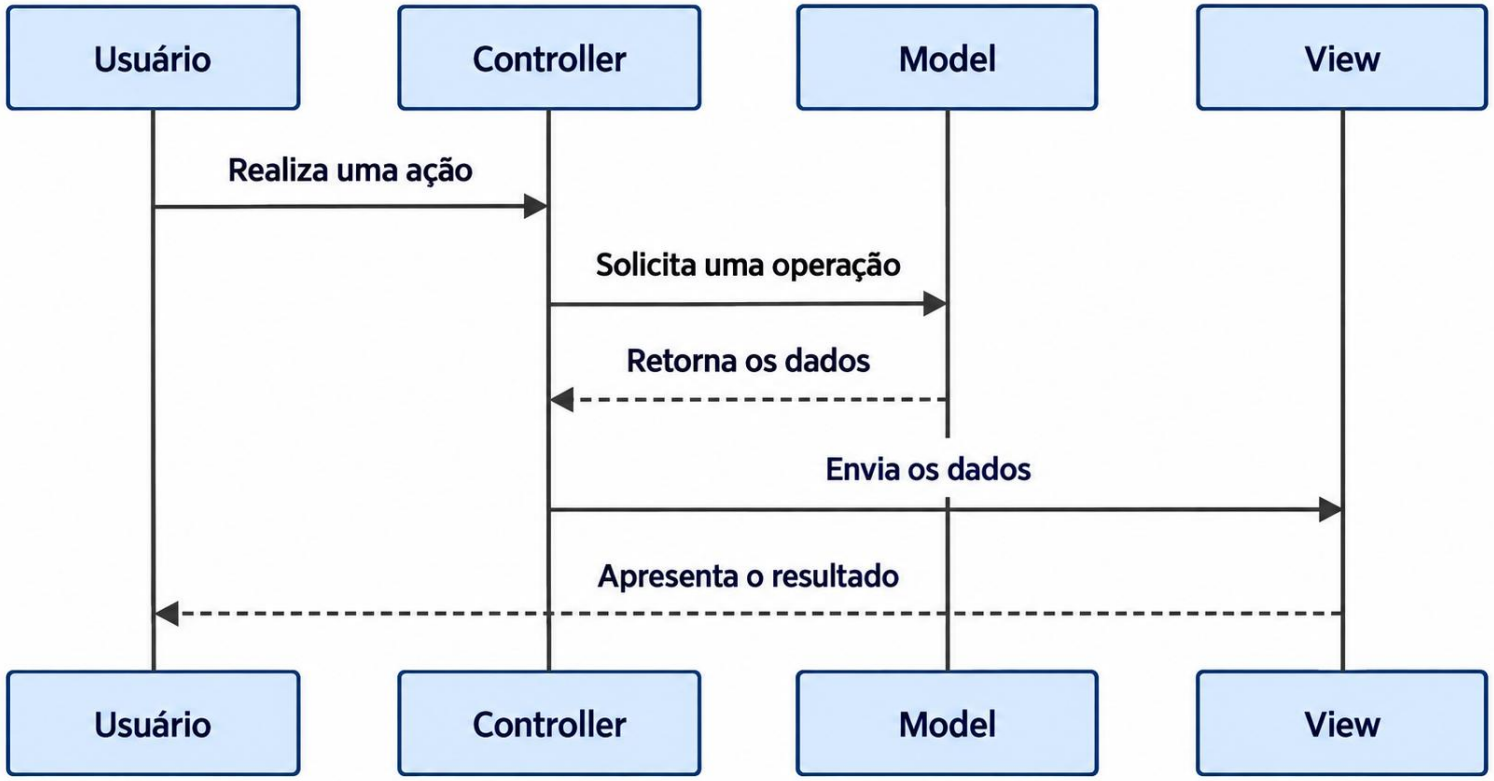
Como o mesmo servidor produz respostas diferentes?

Estrutura geral do projeto

```
mural-servidor-aula-02/
├── package.json
├── server.js
├── app.js
├── models/
├── routes/
├── views/
├── public/
├── database/
└── test/
```

- configuração e comandos
- inicialização do servidor
- configuração do Express
- dados e persistência
- endereços da aplicação
- templates HTML
- CSS público e testes

Percurso de uma requisição



- o navegador envia a requisição
- a rota coordena o processamento
- o Model consulta os dados
- a View produz o HTML
- o servidor envia a resposta

Tecnologias e responsabilidades

- Node.js executa JavaScript no servidor.
- Express recebe requisições e escolhe rotas.
- Sequelize traduz operações JavaScript para o banco.
- SQLite armazena dados no arquivo mural.sqlite.
- Handlebars combina dados e templates HTML.

Qual tecnologia participa de cada etapa do percurso?

O projeto declara seus metadados

```
{
  "name": "mural-servidor-aula-02",
  "version": "1.0.0",
  "private": true,
  "description": "Projeto didático completo..."
}
```

LEITURA DA LINHA

name identifica o pacote. O npm utiliza esse nome para representar o projeto.

EFEITO NO PROJETO

A identificação aparece nos metadados e em ferramentas que leem o package.json.

O name inicia o servidor? Não. Ele apenas identifica o projeto.

private protege o projeto

```
"name": "mural-servidor-aula-02",
"version": "1.0.0",
"private": true
```

LEITURA DA LINHA

private com valor true impede uma publicação acidental no registro público do npm.

EFEITO NO PROJETO

O projeto continua instalável e executável localmente.

private impede npm install? Não.

Scripts transformam comandos em atalhos

```
"scripts": {
  "start": "node server.js",
  "dev": "node --watch server.js",
  "test": "node --test"
}
```

LEITURA DA LINHA

O nome start fica associado à execução de node server.js.

EFEITO NO PROJETO

Ao executar npm start, o npm procura e executa esta instrução.

Qual arquivo inicia a aplicação? server.js.

O modo de desenvolvimento observa mudanças

```
"start": "node server.js",
"dev": "node --watch server.js",
"test": "node --test"
```

LEITURA DA LINHA

--watch observa alterações e reinicia o processo Node durante o desenvolvimento.

EFEITO NO PROJETO

O professor pode alterar o código e observar o servidor reiniciar.

Qual comando utiliza esse script? npm run dev.

O script de testes usa o recurso nativo do Node

```
"start": "node server.js",
"dev": "node --watch server.js",
"test": "node --test"
```

LEITURA DA LINHA

node --test procura e executa os testes do projeto.

EFEITO NO PROJETO

npm test verifica os comportamentos definidos em test/app.test.js.

O teste precisa abrir manualmente o navegador? Não.

As dependências registram bibliotecas externas

```
"dependencies": {
  "express": "4.21.2",
  "express-handlebars": "7.1.3",
  "sequelize": "6.37.7",
  "sqlite3": "5.1.7"
}
```

LEITURA DA LINHA

Express fornece a estrutura HTTP, os roteadores e os middlewares.

EFEITO NO PROJETO

npm install baixa a versão indicada e a coloca em node_modules.

require("express") funciona antes da instalação? Não.

require e module.exports conectam arquivos

- require() carrega o valor exportado por outro módulo.
- Caminhos iniciados por ./ apontam para arquivos do projeto.
- Nomes sem ./ procuram módulos nativos ou dependências.
- module.exports define o que o arquivo disponibiliza.

Cada arquivo possui escopo próprio.

server.js controla a inicialização

- carrega as funções de app.js
- define a porta
- prepara o banco
- cria a aplicação Express
- abre o servidor e trata falhas

Este arquivo coloca a aplicação em funcionamento.

server.js importa duas funções

```
const { createApp, prepararBanco } = require("./app");
```

LEITURA DA LINHA

A desestruturação retira createApp e prepararBanco do objeto exportado por app.js.

EFEITO NO PROJETO

server.js pode chamar as duas funções sem carregar outros nomes.

Por que o caminho começa com ./?
Porque app.js pertence ao projeto.

A porta pode vir do ambiente

```
const PORT = Number(process.env.PORT) || 3000;
```

LEITURA DA LINHA

process.env.PORT consulta uma variável do ambiente. Number tenta convertê-la para número.

EFEITO NO PROJETO

Se a variável não existir, o operador || seleciona 3000.

Qual porta será usada localmente sem configuração? 3000.

A função de inicialização é assíncrona

```
async function iniciarServidor() {
  // preparação e abertura
}
```

LEITURA DA LINHA

async permite utilizar await dentro da função e faz a função devolver uma promise.

EFEITO NO PROJETO

A inicialização pode aguardar tarefas que levam tempo, como conectar ao banco.

Qual operação precisará ser aguardada primeiro? prepararBanco().

try delimita a tentativa de inicialização

```
async function iniciarServidor() {
  try {
    // operações de inicialização
  } catch (erro) {
    // tratamento
  }
}
```

LEITURA DA LINHA

O bloco try reúne as operações que podem falhar durante a preparação.

EFEITO NO PROJETO

Uma falha desvia a execução para catch, evitando encerramento sem mensagem.

Que falha pode ocorrer aqui? Por exemplo, problema ao preparar o banco.

O banco fica pronto antes do servidor

```
try {
  await prepararBanco();
  const app = createApp();
}
```

LEITURA DA LINHA

await pausa esta função até prepararBanco concluir ou lançar um erro.

EFEITO NO PROJETO

O servidor não aceita requisições antes de a conexão e a tabela estarem prontas.

O que aconteceria sem await? A inicialização poderia prosseguir cedo demais.

createApp monta a aplicação

```
await prepararBanco();
const app = createApp();
const servidor = app.listen(PORT, () => { ... });
```

LEITURA DA LINHA

createApp executa as configurações de app.js e devolve a aplicação Express.

EFEITO NO PROJETO

A constante app passa a oferecer métodos como listen.

createApp abre a porta? Não. Essa função apenas configura.

listen abre a porta

```
const servidor = app.listen(PORT, () => {
  console.log(`Mural disponível em http://localhost:${PORT}`);
  console.log("Pressione Ctrl+C para encerrar.");
});
```

LEITURA DA LINHA

app.listen solicita ao sistema operacional que o processo escute a porta escolhida.

EFEITO NO PROJETO

A aplicação passa a receber requisições HTTP.

Quando o navegador já pode acessar o mural? Depois que listen concluir.

A callback confirma que o servidor iniciou

```
app.listen(PORT, () => {
  console.log(`Mural disponível em http://localhost:${PORT}`);
});
```

LEITURA DA LINHA

A função fornecida a listen executa quando a porta já está disponível.

EFEITO NO PROJETO

O terminal informa o endereço correto para acesso.

Por que a função não possui nome? Ela será usada apenas neste ponto.

A referência do servidor é devolvida

```
const servidor = app.listen(PORT, () => { ... });
return servidor;
```

LEITURA DA LINHA

return devolve o objeto que representa o servidor em execução.

EFEITO NO PROJETO

Os testes podem guardar essa referência e fechar o servidor ao final.

Qual método encerrará o servidor nos testes? servidor.close().

catch registra a falha de inicialização

```
} catch (erro) {
  console.error("Não foi possível iniciar o servidor.");
  console.error(erro);
  process.exitCode = 1;
  return null;
}
```

LEITURA DA LINHA

console.error apresenta uma mensagem clara no terminal.

EFEITO NO PROJETO

O desenvolvedor recebe contexto antes dos detalhes técnicos do erro.

Essa mensagem aparece para o usuário no navegador? Não.

exitCode informa encerramento com falha

```
console.error(erro);
process.exitCode = 1;
return null;
```

LEITURA DA LINHA

O código 1 comunica ao sistema operacional que o processo terminou com erro.

EFEITO NO PROJETO

Ferramentas e scripts conseguem detectar a falha da execução.

Por que evitar process.exit() imediato? Para permitir encerramento controlado.

O servidor inicia apenas na execução direta

```
if (require.main === module) {
  iniciarServidor();
}
```

LEITURA DA LINHA

require.main === module verifica se server.js foi executado diretamente.

EFEITO NO PROJETO

Ao importar o arquivo em outro módulo, o servidor não inicia automaticamente.

Qual situação usa essa proteção? Testes ou reutilização do módulo.

server.js exporta a função de inicialização

```
module.exports = { iniciarServidor };
```

LEITURA DA LINHA

O arquivo disponibiliza iniciarServidor para outros módulos.

EFEITO NO PROJETO

Outro arquivo pode importar e controlar a inicialização.

O que require("./server") receberá? Um objeto com iniciarServidor.

app.js configura a aplicação

- importa bibliotecas e módulos locais
- configura o Handlebars
- registra middlewares
- monta os roteadores
- define respostas 404 e 500
- prepara e povoa o banco

app.js não abre a porta.

path constrói caminhos compatíveis

```
const path = require("node:path");
```

LEITURA DA LINHA

node:path é um módulo nativo do Node. Ele manipula caminhos de arquivos.

EFEITO NO PROJETO

path.join evita montar caminhos manualmente com barras específicas.

Foi necessário instalar node:path? Não. Ele já pertence ao Node.

Express entra como dependência

```
const express = require("express");
```

LEITURA DA LINHA

require procura o pacote Express instalado em node_modules.

EFEITO NO PROJETO

A constante express recebe a função principal da biblioteca.

O que acontecerá em express()? A aplicação será criada.

engine é retirada do express-handlebars

```
const { engine } = require("express-handlebars");
```

LEITURA DA LINHA

A desestruturação obtém apenas a função engine exportada pelo pacote.

EFEITO NO PROJETO

Essa função configurará como arquivos .hbs serão renderizados.

engine é uma View? Não. Ela configura o mecanismo de templates.

Model e conexão vêm da pasta models

```
const { sequelize, Publicacao } = require("./models");
```

LEITURA DA LINHA

models/index.js exporta a conexão sequelize e o Model Publicacao.

EFEITO NO PROJETO

app.js pode preparar o banco e inserir dados iniciais.

Qual item representa a conexão?
sequelize.

Os roteadores chegam como módulos

```
const indexRouter = require("./routes/index");
const publicacoesRouter = require("./routes/publicacoes");
```

LEITURA DA LINHA

indexRouter contém as rotas gerais, como / e /saude.

EFEITO NO PROJETO

As rotas permanecem separadas do arquivo principal de configuração.

Qual roteador cuidará de /publicacoes/1? publicacoesRouter.

express() cria o objeto app

```
function createApp() {
  const app = express();
  // configurações
  return app;
}
```

LEITURA DA LINHA

A chamada express() cria uma aplicação que receberá configurações e rotas.

EFEITO NO PROJETO

app passa a disponibilizar engine, set, use e listen.

O servidor já escuta a porta neste ponto? Não.

app.engine associa hbs ao Handlebars

```
app.engine(
  "hbs",
  engine({ ... })
);
```

LEITURA DA LINHA

app.engine registra uma função capaz de transformar templates em HTML.

EFEITO NO PROJETO

Quando res.render selecionar uma View .hbs, o Express usará essa engine.

Qual extensão foi associada? hbs.

extname define a extensão dos templates

```
engine({
  extname: ".hbs",
  defaultLayout: "main"
})
```

LEITURA DA LINHA

extname informa que os templates utilizam a extensão .hbs.

EFEITO NO PROJETO

O projeto pode chamar res.render("index") sem escrever .hbs.

Qual arquivo será procurado?
views/index.hbs.

defaultLayout define a moldura

```
engine({
  extname: ".hbs",
  defaultLayout: "main"
})
```

LEITURA DA LINHA

main será aplicado como layout padrão às Views renderizadas.

EFEITO NO PROJETO

Cabeçalho, menu e rodapé não precisam ser repetidos em cada página.

Onde está esse arquivo?
views/layouts/main.hbs.

layoutsDir informa a pasta dos layouts

```
layoutsDir: path.join(__dirname, "views", "layouts"),
partialsDir: path.join(__dirname, "views", "partials"),
```

LEITURA DA LINHA

__dirname representa a pasta em que app.js está localizado.

EFEITO NO PROJETO

path.join produz o caminho absoluto da pasta views/layouts.

Por que usar caminho absoluto? Para evitar depender da pasta do terminal.

partialsDir informa a pasta reutilizável

```
layoutsDir: path.join(__dirname, "views", "layouts"),
partialsDir: path.join(__dirname, "views", "partials"),
```

LEITURA DA LINHA

partialsDir aponta para pequenos templates reutilizados em outras Views.

EFEITO NO PROJETO

{{> cartao-publicacao}} encontra o arquivo do cartão.

O partial possui o HTML completo da página? Não.

helpers adiciona funções aos templates

```
helpers: {
  formatarData(valor) {
    // transformação
  }
}
```

LEITURA DA LINHA

formatarData recebe um valor quando o template chama o helper.

EFEITO NO PROJETO

A View pode exibir uma data em formato brasileiro.

Onde o helper é chamado? Na View de detalhes.

Intl.DateTimeFormat prepara o formato

```
return new Intl.DateTimeFormat("pt-BR", {
  dateStyle: "short"
}).format(new Date(valor));
```

LEITURA DA LINHA

Intl.DateTimeFormat recebe a localidade pt-BR e opções de apresentação.

EFEITO NO PROJETO

Dia, mês e ano seguem a convenção brasileira.

O helper altera o valor armazenado no banco? Não.

format converte a data para texto

```
return new Intl.DateTimeFormat("pt-BR", {
  dateStyle: "short"
}).format(new Date(valor));
```

LEITURA DA LINHA

new Date(valor) cria um objeto Date. format devolve uma string legível.

EFEITO NO PROJETO

createdAt pode aparecer como 24/09/2026.

O retorno continua sendo um objeto Date? Não. É texto.

view engine define o mecanismo padrão

```
app.set("view engine", "hbs");
app.set("views", path.join(__dirname, "views"));
```

LEITURA DA LINHA

O Express passa a tratar hbs como mecanismo padrão de renderização.

EFEITO NO PROJETO

res.render não precisa informar manualmente a engine.

Qual chamada renderiza uma View?
res.render().

views define a pasta de templates

```
app.set("view engine", "hbs");
app.set("views", path.join(__dirname, "views"));
```

LEITURA DA LINHA

A configuração informa onde o Express deve procurar as Views.

EFEITO NO PROJETO

res.render("publicacoes/listar") procura dentro de views/publicacoes.

A rota precisa escrever views/ no caminho? Não.

urlencoded interpreta formulários

```
app.use(express.urlencoded({ extended: false }));
app.use(express.json());
```

LEITURA DA LINHA

O middleware interpreta corpos enviados no formato de formulário.

EFEITO NO PROJETO

Em futuras rotas POST, os dados ficarão disponíveis em req.body.

A listagem atual usa req.body? Não.

json interpreta corpos em JSON

```
app.use(express.urlencoded({ extended: false }));
app.use(express.json());
```

LEITURA DA LINHA

O middleware lê JSON recebido no corpo da requisição.

EFEITO NO PROJETO

Express poderá disponibilizar o objeto convertido em req.body.

Este middleware devolve JSON? Não. Ele interpreta JSON recebido.

static entrega arquivos públicos

```
app.use(express.static(path.join(__dirname, "public")));
```

LEITURA DA LINHA

express.static procura arquivos dentro da pasta public.

EFEITO NO PROJETO

O navegador consegue solicitar /stylesheets/style.css.

Por que a URL não contém /public? A pasta já virou a raiz pública.

O roteador geral ocupa a raiz

```
app.use("/", indexRouter);
app.use("/publicacoes", publicacoesRouter);
```

LEITURA DA LINHA

O prefixo / monta indexRouter na raiz da aplicação.

EFEITO NO PROJETO

As rotas / e /saude preservam seus endereços.

Qual arquivo define /saude?
routes/index.js.

O roteador de publicações recebe um prefixo

```
app.use("/", indexRouter);
app.use("/publicacoes", publicacoesRouter);
```

LEITURA DA LINHA

O prefixo /publicacoes antecede todas as rotas desse roteador.

EFEITO NO PROJETO

A rota interna /:id torna-se /publicacoes/:id.

Qual endereço ativa a rota interna /? /publicacoes.

O middleware 404 recebe o que sobrou

```
app.use((req, res) => {
  res.status(404).render("404", {
    titulo: "Página não encontrada",
    caminho: req.originalUrl
  });
});
```

LEITURA DA LINHA

Sem caminho específico, este app.use aceita qualquer requisição que chegou até ele.

EFEITO NO PROJETO

Como aparece depois das rotas, representa um endereço não atendido.

Por que ele deve ficar depois dos roteadores? Para não bloquear rotas válidas.

status 404 acompanha a página

```
res.status(404).render("404", {
  titulo: "Página não encontrada",
  caminho: req.originalUrl
});
```

LEITURA DA LINHA

res.status(404) define o código HTTP antes de renderizar a View.

EFEITO NO PROJETO

Navegador e ferramentas reconhecem que o recurso não foi encontrado.

Mostrar uma mensagem sem status 404 seria suficiente? Não para o protocolo HTTP.

originalUrl registra o endereço solicitado

```
titulo: "Página não encontrada",
caminho: req.originalUrl
```

LEITURA DA LINHA

req.originalUrl contém o caminho originalmente solicitado pelo navegador.

EFEITO NO PROJETO

A página 404 informa exatamente qual endereço não está disponível.

Qual valor aparece ao acessar /nao-existe? /nao-existe.

O middleware de erro usa quatro parâmetros

```
app.use((erro, req, res, next) => {
  // tratamento centralizado
});
```

LEITURA DA LINHA

A assinatura com erro, req, res e next identifica um middleware de erro no Express.

EFEITO NO PROJETO

Erros enviados por next(erro) chegam a esta função.

Uma função com apenas req e res receberia esses erros? Não.

headersSent evita uma segunda resposta

```
if (res.headersSent) {
  return next(erro);
}
```

LEITURA DA LINHA

res.headersSent indica que a aplicação já começou a enviar a resposta.

EFEITO NO PROJETO

O código evita tentar enviar outra página e encaminha o erro adiante.

Por que há return? Para interromper a função atual.

A mensagem 500 depende do ambiente

```
mensagem:
  process.env.NODE_ENV === "development"
    ? erro.message
    : "Não foi possível concluir a solicitação."
```

LEITURA DA LINHA

A condição verifica se a aplicação está em modo de desenvolvimento.

EFEITO NO PROJETO

O desenvolvedor pode ver detalhes, enquanto o usuário recebe mensagem segura.

Por que ocultar detalhes em produção? Para não expor informações internas.

createApp devolve a aplicação configurada

```
return app;
}
```

LEITURA DA LINHA

Depois de registrar engine, middlewares e rotas, a função devolve app.

EFEITO NO PROJETO

server.js pode chamar listen e os testes podem usar uma porta aleatória.

Qual benefício principal? Reutilização da mesma configuração.

prepararBanco organiza a persistência

- testa a conexão
- sincroniza a tabela
- conta os registros
- insere dados quando o banco está vazio

Todas as operações precisam ser aguardadas.

authenticate testa a conexão

```
async function prepararBanco() {
  await sequelize.authenticate();
  await sequelize.sync();
}
```

LEITURA DA LINHA

authenticate tenta estabelecer comunicação com o SQLite configurado.

EFEITO NO PROJETO

Uma falha impede que o servidor prossiga silenciosamente.

Onde o erro será capturado? No try/catch de iniciarServidor.

sync cria a estrutura necessária

```
await sequelize.authenticate();
await sequelize.sync();
```

LEITURA DA LINHA

sync compara os Models com as tabelas disponíveis e cria o necessário.

EFEITO NO PROJETO

A tabela publicacoes pode surgir na primeira execução.

sync insere as seis publicações? Não.

count verifica quantos registros existem

```
const quantidade = await Publicacao.count();
if (quantidade === 0) {
  // inserir dados
}
```

LEITURA DA LINHA

Publicacao.count consulta a quantidade de registros da tabela.

EFEITO NO PROJETO

A constante quantidade orienta a decisão seguinte.

count devolve um array? Não. Devolve um número.

O seed ocorre somente com tabela vazia

```
if (quantidade === 0) {
  await Publicacao.bulkCreate([ ... ]);
}
```

LEITURA DA LINHA

A comparação estrita verifica se nenhum registro foi encontrado.

EFEITO NO PROJETO

Reiniciar o servidor não duplica os dados iniciais.

O que acontece quando quantidade vale 6? O bloco não executa.

bulkCreate insere vários objetos

```
await Publicacao.bulkCreate([
  { titulo, conteudo, autor, categoria },
  // outros objetos
]);
```

LEITURA DA LINHA

bulkCreate recebe um array de objetos que representam novos registros.

EFEITO NO PROJETO

Seis publicações passam a existir no SQLite.

Por que há await? A inserção é assíncrona.

models/index.js centraliza a conexão

- cria uma instância do Sequelize
- define SQLite como dialeto
- informa o arquivo do banco
- carrega o Model Publicacao
- exporta conexão e Model

Rotas importam o Model sem recriar a conexão.

new Sequelize cria a conexão

```
const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: ...
});
```

LEITURA DA LINHA

new instancia Sequelize com as opções fornecidas.

EFEITO NO PROJETO

sequelize passa a representar a conexão e oferecer métodos de persistência.

sequelize é a tabela? Não. É a conexão configurada.

dialect escolhe a linguagem do banco

```
dialect: "sqlite",
storage: path.join(...)
```

LEITURA DA LINHA

dialect informa qual sistema de banco o Sequelize deve utilizar.

EFEITO NO PROJETO

O ORM gera comandos compatíveis com SQLite.

Esse valor seria igual em um projeto MySQL? Não.

storage aponta para mural.sqlite

```
storage: path.join(__dirname, "..", "database", "mural.sqlite"),
```

LEITURA DA LINHA

O caminho sai de models, volta uma pasta e entra em database.

EFEITO NO PROJETO

Os dados persistem no arquivo database/mural.sqlite.

Fechar o servidor apaga esse arquivo? Não.

logging false reduz ruído no terminal

logging: false

LEITURA DA LINHA

A opção desativa a exibição automática de cada comando SQL executado.

EFEITO NO PROJETO

O terminal mostra apenas mensagens importantes para a aula.

As consultas deixam de acontecer? Não.

O arquivo publicacao funciona como fábrica

```
const Publicacao = require("./publicacao")(sequelize, DataTypes);
```

LEITURA DA LINHA

require carrega uma função e os parênteses seguintes executam essa função.

EFEITO NO PROJETO

A fábrica recebe a conexão e os tipos e devolve o Model configurado.

Por que existem dois pares de parênteses? Um importa e o outro executa.

define cria o Model Publicacao

```
return sequelize.define(
  "Publicacao",
  { /* atributos */ },
  { /* opções */ }
);
```

LEITURA DA LINHA

sequelize.define recebe nome, atributos e opções do Model.

EFEITO NO PROJETO

Publicacao ganha métodos como findAll, findByPk e count.

O Model representa um registro específico? Ele representa a entidade e suas operações.

id é a chave primária

```
id: {
  type: DataTypes.INTEGER,
  autoIncrement: true,
  primaryKey: true
}
```

LEITURA DA LINHA

primaryKey identifica o campo que diferencia cada registro.

EFEITO NO PROJETO

findByPk pode localizar uma publicação pelo id.

Dois registros podem compartilhar a mesma chave primária? Não.

autoIncrement gera o próximo id

```
id: {
  type: DataTypes.INTEGER,
  autoIncrement: true,
  primaryKey: true
}
```

LEITURA DA LINHA

autoIncrement solicita ao banco a geração sequencial do identificador.

EFEITO NO PROJETO

O código não precisa informar id nos objetos do bulkCreate.

Qual id tende a receber o primeiro registro? 1.

titulo possui tipo e limite

```
titulo: {
  type: DataTypes.STRING(120),
  allowNull: false,
  validate: { notEmpty: true, len: [3, 120] }
}
```

LEITURA DA LINHA

STRING(120) representa texto curto com limite de 120 caracteres.

EFEITO NO PROJETO

O Sequelize utiliza essa informação ao validar e criar a coluna.

conteúdo longo deveria usar o mesmo tipo? O projeto utiliza TEXT.

allowNull exige um valor

```
titulo: {
  allowNull: false,
  validate: { notEmpty: true }
}
```

LEITURA DA LINHA

allowNull false impede que o campo receba null.

EFEITO NO PROJETO

A persistência rejeita uma publicação sem título.

null e string vazia significam a mesma coisa? Não.

notEmpty rejeita texto vazio

```
validate: {
  notEmpty: true,
  len: [3, 120]
}
```

LEITURA DA LINHA

notEmpty exige conteúdo textual, além de o valor não ser nulo.

EFEITO NO PROJETO

Uma string vazia não atende à validação.

Qual validação controla o comprimento?
len.

conteudo usa DataTypes.TEXT

```
conteudo: {
  type: DataTypes.TEXT,
  allowNull: false,
  validate: { notEmpty: true }
}
```

LEITURA DA LINHA

TEXT serve para conteúdos maiores do que um título ou categoria.

EFEITO NO PROJETO

Cada publicação pode armazenar uma explicação mais extensa.

O conteúdo pode ficar vazio? Não.

autor e categoria possuem valores padrão

```
autor: {
  defaultValue: "Estudante"
},
categoria: {
  defaultValue: "Geral"
}
```

LEITURA DA LINHA

defaultValue fornece Estudante quando o código não envia um autor.

EFEITO NO PROJETO

O registro recebe um valor válido sem exigir preenchimento explícito.

Qual padrão será usado para categoria? Geral.

tableName fixa o nome da tabela

```
{
  tableName: "publicacoes",
  timestamps: true
}
```

LEITURA DA LINHA

tableName define explicitamente o nome físico da tabela.

EFEITO NO PROJETO

O Sequelize utiliza publicacoes no SQLite.

O nome do Model continua sendo Publicacao? Sim.

timestamps cria datas automáticas

```
tableName: "publicacoes",
timestamps: true
```

LEITURA DA LINHA

timestamps adiciona createdAt e updatedAt ao Model.

EFEITO NO PROJETO

A View consegue exibir a data de criação sem um campo manual.

Qual helper formata createdAt?
formatarData.

routes/index.js trata respostas gerais

- cria um Router
- renderiza a página inicial
- oferece o endpoint /saude
- exporta o roteador

O prefixo final vem de app.js.

express.Router cria um roteador

```
const express = require("express");
const router = express.Router();
```

LEITURA DA LINHA

Router cria um conjunto independente de rotas.

EFEITO NO PROJETO

O arquivo pode organizar endereços sem acessar diretamente app.

Quem monta esse roteador na aplicação? app.js.

router.get responde ao método GET

```
router.get("/", (req, res) => {
  // resposta
});
```

LEITURA DA LINHA

get associa o caminho / a uma função executada em requisições GET.

EFEITO NO PROJETO

Com o prefixo / de app.js, esta rota responde à página inicial.

Qual método seria usado para criar dados? Normalmente POST.

res.render escolhe a View inicial

```
res.render("index", {
  titulo: "Mural de Publicações",
  subtitulo: "Node.js, Express, Handlebars, Sequelize e SQLite"
});
```

LEITURA DA LINHA

render procura views/index.hbs e envia o objeto de dados.

EFEITO NO PROJETO

titulo e subtitulo ficam disponíveis no template.

O navegador recebe o arquivo .hbs? Não. Recebe HTML.

A rota de saúde devolve JSON

```
router.get("/saude", (req, res) => {
  res.json({ status: "ok", aplicacao: "mural-servidor-aula-02" });
});
```

LEITURA DA LINHA

res.json serializa o objeto e define a resposta como JSON.

EFEITO NO PROJETO

Ferramentas podem verificar rapidamente se a aplicação responde.

Essa rota usa uma View? Não.

O roteador precisa ser exportado

```
module.exports = router;
```

LEITURA DA LINHA

O arquivo disponibiliza o Router configurado.

EFEITO NO PROJETO

app.js consegue importá-lo como indexRouter.

Sem exportação, app.js receberia as rotas? Não.

routes/publicacoes.js acessa o Model

- importa Publicacao
- lista todos os registros
- procura um registro pelo id
- renderiza páginas específicas
- encaminha falhas ao middleware

As callbacks exercem a responsabilidade de Controller.

A rota de listagem é assíncrona

```
router.get("/", async (req, res, next) => {
  try {
    // consulta e renderização
  } catch (erro) {
    next(erro);
  }
});
```

LEITURA DA LINHA

async permite aguardar a consulta ao banco dentro da callback.

EFEITO NO PROJETO

A rota pode usar await Publicacao.findAll.

Por que aparece next? Para encaminhar erros.

findAll consulta a coleção

```
const publicacoes = await Publicacao.findAll({
  order: [["createdAt", "DESC"]],
  raw: true
});
```

LEITURA DA LINHA

findAll consulta vários registros e devolve um array.

EFEITO NO PROJETO

publicacoes conterá objetos ou será um array vazio.

findAll devolve apenas um registro? Não.

order define a ordem dos registros

```
order: [["createdAt", "DESC"]],
raw: true
```

LEITURA DA LINHA

A ordenação usa createdAt em ordem decrescente.

EFEITO NO PROJETO

Publicações mais recentes aparecem antes das antigas.

Qual valor indicaria ordem crescente?
ASC.

raw true devolve objetos simples

```
order: [["createdAt", "DESC"]],
raw: true
```

LEITURA DA LINHA

raw true solicita objetos JavaScript simples em vez de instâncias do Model.

EFEITO NO PROJETO

O Handlebars recebe propriedades diretas com mais facilidade.

As propriedades ainda incluem titulo e autor? Sim.

A rota envia dados calculados à View

```
res.render("publicacoes/listar", {
  titulo: "Publicações da turma",
  publicacoes,
  possuiPublicacoes: publicacoes.length > 0,
  quantidade: publicacoes.length
});
```

LEITURA DA LINHA

A forma abreviada publicacoes equivale a publicacoes: publicacoes.

EFEITO NO PROJETO

A View recebe o array retornado pelo banco.

Qual propriedade informa o total?
quantidade.

possuiPublicacoes prepara uma decisão

```
possuiPublicacoes: publicacoes.length > 0,
quantidade: publicacoes.length
```

LEITURA DA LINHA

A comparação produz true quando o array contém pelo menos um item.

EFEITO NO PROJETO

O template pode usar {{#if possuiPublicacoes}}.

Qual valor surge para um array vazio? false.

catch encaminha o erro

```
} catch (erro) {
  next(erro);
}
```

LEITURA DA LINHA

next recebe o erro capturado e procura o middleware de erro.

EFEITO NO PROJETO

A rota não precisa repetir toda a lógica da página 500.

Onde o middleware central foi registrado?
No final de app.js.

O parâmetro :id ocupa parte da URL

```
router.get("/:id", async (req, res, next) => {
  // processamento
});
```

LEITURA DA LINHA

:id representa um segmento variável do endereço.

EFEITO NO PROJETO

/publicacoes/3 disponibiliza o texto 3 em req.params.id.

O nome id precisa combinar com req.params.id? Sim.

findByPk procura pela chave primária

```
const publicacao = await Publicacao.findByPk(
  req.params.id,
  { raw: true }
);
```

LEITURA DA LINHA

findByPk procura um único registro usando a chave primária.

EFEITO NO PROJETO

O retorno será um objeto encontrado ou null.

Qual campo foi definido como chave primária? id.

A rota trata a ausência do registro

```
if (!publicacao) {
  return res.status(404).render("404", {
    titulo: "Publicação não encontrada"
  });
}
```

LEITURA DA LINHA

!publicacao será true quando findByPk devolver null.

EFEITO NO PROJETO

A aplicação apresenta 404 em vez de tentar usar um objeto inexistente.

Por que usar return? Para interromper o restante da callback.

A View recebe uma única publicação

```
return res.render("publicacoes/detalhar", {
  titulo: publicacao.titulo,
  publicacao
});
```

LEITURA DA LINHA

A rota seleciona a View detalhar dentro da pasta publicacoes.

EFEITO NO PROJETO

O objeto fica acessível como publicacao no template.

A View precisará usar publicacao.titulo ou apenas titulo? Para o objeto, publicacao.titulo.

main.hbs fornece a estrutura comum

- declara o documento HTML
- define metadados e CSS
- apresenta cabeçalho e menu
- recebe a View em {{{body}}}
- apresenta o rodapé

O layout envolve o conteúdo específico.

{{titulo}} preenche o title

```
<title>{{titulo}} | Aula 02</title>
<link rel="stylesheet" href="/stylesheets/style.css">
```

LEITURA DA LINHA

Handlebars substitui titulo pelo valor enviado em res.render.

EFEITO NO PROJETO

Cada página apresenta um título coerente na aba do navegador.

De onde vem titulo na página de detalhes? Da rota.

Três chaves inserem o HTML da View

```
<main class="limite conteudo-principal">
  {{{body}}}
</main>
```

LEITURA DA LINHA

body contém o HTML produzido pela View específica.

EFEITO NO PROJETO

O conteúdo entra entre o cabeçalho e o rodapé do layout.

Por que três chaves? Para inserir HTML sem escapar as tags.

A listagem escolhe entre dois estados

```
{{#if possuiPublicacoes}}
  <!-- grade de cartões -->
{{else}}
  <!-- estado vazio -->
{{/if}}
```

LEITURA DA LINHA

#if avalia o booleano preparado pela rota.

EFEITO NO PROJETO

A interface evita apresentar uma grade vazia sem explicação.

Quem calculou possuiPublicacoes? A rota.

each percorre o array

```
{{#each publicacoes}}
  {{> cartao-publicacao}}
{{/each}}
```

LEITURA DA LINHA

#each executa o conteúdo uma vez para cada objeto do array.

EFEITO NO PROJETO

Seis registros produzem seis cartões.

Qual é o contexto dentro do loop? A publicação atual.

O partial cria um cartão por item

```
<span class="categoria">{{categoria}}</span>
<h2>{{titulo}}</h2>
<p>{{conteudo}}</p>
<span>{{autor}}</span>
```

LEITURA DA LINHA

{{titulo}} procura a propriedade no objeto atual do each.

EFEITO NO PROJETO

O mesmo arquivo apresenta títulos diferentes em cada repetição.

O partial precisa receber parâmetros manualmente? O contexto já vem do each.

O id constrói o link de detalhes

```
<a href="/publicacoes/{{id}}">
  Ler publicação
</a>
```

LEITURA DA LINHA

Handlebars combina o trecho fixo da URL com o id atual.

EFEITO NO PROJETO

O cartão 3 aponta para /publicacoes/3.

Qual rota receberá esse clique? router.get('/:id').

A View de detalhes usa propriedades aninhadas

```
{{publicacao.categoria}}
{{publicacao.titulo}}
{{publicacao.conteudo}}
{{publicacao.autor}}
```

LEITURA DA LINHA

publicacao identifica o objeto enviado pela rota. titulo acessa uma propriedade.

EFEITO NO PROJETO

A página apresenta os dados do registro encontrado.

Por que não existe each aqui? Há apenas um objeto.

O helper aparece dentro do template

```
Publicada em {{formatarData publicacao.createdAt}}
```

LEITURA DA LINHA

O primeiro nome identifica o helper. O segundo trecho fornece o argumento.

EFEITO NO PROJETO

A data completa do banco aparece em formato curto brasileiro.

Qual função recebe createdAt?
formatarData.

O CSS chega por express.static

```
<link rel="stylesheet" href="/stylesheets/style.css">
```

```
app.use(express.static(path.join(__dirname, "public")));
```

LEITURA DA LINHA

O navegador solicita a URL indicada pelo atributo href.

EFEITO NO PROJETO

O middleware encontra public/stylesheets/style.css e envia o arquivo.

Quem executa o CSS? O navegador.

A grade responde ao tamanho da tela

```
.grade {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
@media (max-width: 600px) {
  .grade { grid-template-columns: 1fr; }
}
```

LEITURA DA LINHA

Em telas amplas, a grade cria três colunas de mesma proporção.

EFEITO NO PROJETO

Os cartões ocupam o espaço disponível de maneira organizada.

Quantas colunas aparecem abaixo de 600px? Uma.

Os testes iniciam a aplicação em porta aleatória

```
test.before(async () => {
  await prepararBanco();
  servidor = createApp().listen(0);
  baseUrl = `http://127.0.0.1:${servidor.address().port}`;
});
```

LEITURA DA LINHA

A porta 0 solicita ao sistema uma porta livre automaticamente.

EFEITO NO PROJETO

Os testes evitam conflito com a porta 3000.

A aplicação testada usa a mesma createApp? Sim.

Um teste faz uma requisição real

```
const resposta = await fetch(`${baseUrl}/publicacoes`);
const html = await resposta.text();
assert.equal(resposta.status, 200);
assert.match(html, /Publicações da turma/);
```

LEITURA DA LINHA

fetch envia uma requisição HTTP para o servidor iniciado pelo teste.

EFEITO NO PROJETO

O teste percorre middlewares, rota, banco e renderização.

Esse teste verifica apenas uma função isolada? Não.

Asserções comparam resultado e expectativa

```
assert.equal(resposta.status, 200);
assert.match(html, /Publicações da turma/);
```

LEITURA DA LINHA

assert.equal exige que o status seja exatamente 200.

EFEITO NO PROJETO

O teste falha quando a rota deixa de responder com sucesso.

O que assert.match verifica? Se o HTML contém o padrão esperado.

O percurso completo de /publicacoes

- Express recebe GET /publicacoes.
- publicacoesRouter seleciona a rota interna /.
- findAll consulta a tabela publicacoes.
- res.render envia o array para listar.hbs.
- each aplica o partial a cada objeto.
- main.hbs envolve o conteúdo e o servidor envia HTML.

Cada arquivo participa sem assumir todas as responsabilidades.

O percurso completo de /publicacoes/1

- A URL fornece id igual a 1.
- req.params.id entrega o valor à rota.
- findByPk procura a chave primária.
- A ausência produz 404.
- O objeto encontrado segue para detalhar.hbs.
- formatarData prepara createdAt para exibição.

Trocar 1 por 999 altera a decisão após a consulta.

A organização do projeto se aproxima de MVC

- Model: models/, Sequelize e regras dos dados.
- View: views/ e Handlebars.
- Controller: callbacks implementadas em routes/.
- app.js configura a infraestrutura.
- server.js controla a inicialização.

O projeto ainda não possui uma pasta controllers/ separada.

Estratégia de investigação de falhas

- Servidor não inicia: examine server.js e prepararBanco.
- Rota não responde: confira prefixo e caminho interno.
- Dados não aparecem: siga rota, Model e banco.
- Template falha: confira nomes enviados em res.render.
- CSS não carrega: confira express.static e href.
- Erro inesperado: leia o terminal e o middleware 500.

O percurso da requisição também orienta a depuração.

Síntese da aula

- server.js inicia.
- app.js configura.
- routes/ coordena.
- models/ consulta.
- views/ apresenta.
- public/ estiliza.
- test/ verifica.

Explique /publicacoes/1 sem consultar o código.